

Evaluating VADAR on the LEGO-Puzzles Benchmark

McNair Research

CSE 199 - February 2026

1. Overview

This report evaluates VADAR, an agentic spatial reasoning system, on the LEGO-Puzzles benchmark for multi-step spatial reasoning in multimodal large language models (MLLMs). VADAR uses a pipeline of AI agents (Signature Agent, API Agent, Program Agent) combined with vision models (GroundingDINO, SAM2, UniDepth) to dynamically compose programs that answer spatial reasoning questions. We compare its performance against GPT-4o Mini and Gemini-2.0-Flash, which were evaluated as direct API-based solvers.

The LEGO-Puzzles benchmark contains 220 images and 1,100 multiple-choice questions across 11 spatial reasoning categories: adjacency, height, rotation, rotation status, multi-view, position, ordering, next step, backwards, dependency, and outlier.

2. Experimental Setup

VADAR was deployed on a remote Linux machine (elcap.ucsd.edu) equipped with an NVIDIA RTX 2080 Ti GPU (11 GB VRAM) running CUDA driver 11.4. The system used PyTorch 2.1.2 with CUDA 11.8 toolkit, and GPT-4o as the backbone LLM for all agentic reasoning steps. Several environment-level patches were required to achieve compatibility:

Key Setup Challenges

- CUDA/PyTorch version mismatch: The system nvcc was version 10.2 while PyTorch required CUDA 11.8. GroundingDINO's C++ CUDA extensions could not compile, requiring a fallback patch to use pure Python multi-scale deformable attention.
- bfloat16 unsupported: VADAR assumed bfloat16 precision (Ampere+ GPUs). The RTX 2080 Ti (Turing architecture) required dynamic detection to fall back to float16.
- UniDepth JIT incompatibility: Python 3.10 union type syntax (int | tuple) inside @torch.jit.script decorators caused runtime failures. The decorators were removed.
- SAM2 dependency conflict: Installing SAM2 pulled in PyTorch 2.5+, breaking CUDA compatibility. PyTorch was force-downgraded back to 2.1.2+cu118.

3. Results

Category	Gemini-2.0-Flash	GPT-4o Mini	VADAR (GPT-4o)
Overall	54.18%	23.27%	11.91%
adjacency	65%	57%	58%
height	35%	29%	40%
multi_view	48%	17%	10%
rotation	49%	12%	8%
rotation_status	54%	44%	8%
backwards	57%	17%	7%
dependency	82%	36%	0%
next_step	68%	16%	0%
ordering	46%	4%	0%
outlier	45%	14%	0%
position	47%	10%	0%

VADAR achieved an overall accuracy of 11.91% on the LEGO-Puzzles-Lite benchmark. While this is lower than both baselines overall, performance varies significantly by category.

4. Analysis

Where VADAR Succeeds

VADAR performs competitively on single-image spatial perception tasks. On adjacency (58%), it beats GPT-4o Mini (57%) and approaches Gemini Flash (65%). On height (40%), it is the top performer, beating both Gemini Flash (35%) and GPT-4o Mini (29%). These categories benefit from VADAR's ability to compose vision tools -- using GroundingDINO for object detection and GPT-4o for holistic visual question answering on images that already contain colored markers (red/blue rectangles and arrows).

Where VADAR Fails

VADAR scores 0% on five categories: dependency, next_step, ordering, outlier, and position. These all require multi-image reasoning -- comparing a sequence of assembly step images to determine order, predict what comes next, or identify which step doesn't belong. VADAR's execution engine provides only a single 'image' variable to programs, so any question requiring multiple images causes the generated code to crash with NameError (e.g., 'name x_0 is not defined'). This is a fundamental architectural limitation, not a tuning issue.

Trace Analysis

VADAR provides full intermediate traces at every pipeline stage (signature generation, API generation, program generation, and execution). Examining these traces reveals distinct failure modes:

- For single-image tasks (height, adjacency): The pipeline works as intended. The Program Agent generates correct code using `vqa()` or `_check_adjacency()`, and the execution engine runs it successfully.

- For multi-image tasks (position, ordering, next_step): The Program Agent generates code referencing variables like `x_0`, `x_1`, `x_2` and functions like `_determine_installation_point()` that do not exist in the execution environment. Every such question fails with a runtime error.
- For rotation tasks: The Program Agent attempts to use VQA to compare two images, but the engine only passes one image, leading to inaccurate guesses.

5. Performance vs. Cost

Metric	Gemini-2.0-Flash	GPT-4o Mini	VADAR (GPT-4o)
Runtime	~5 min	~10 min	~4 hours
API calls/question	1	1	3-8
Overall accuracy	54.18%	23.27%	11.91%
Best category	dependency (82%)	adjacency (57%)	adjacency (58%)

VADAR's agentic pipeline is significantly more expensive to run. Each question requires multiple GPT-4o API calls across three agents (signature, API, program generation), plus additional calls during execution for VQA-based modules. Questions that trigger retries (up to 5 attempts) can take 30-45 seconds each. The total benchmark runtime was approximately 4 hours, compared to minutes for direct API calls to GPT-4o Mini or Gemini Flash.

6. Conclusion

VADAR actually does pretty well on single-image stuff like figuring out which LEGO piece is higher or which ones are next to each other, and it even beats GPT-4o Mini and Gemini on some of those. But most of the LEGO benchmark is about looking at a sequence of assembly steps and figuring out what comes next or what order they go in, and VADAR just can't do that since it only looks at one image at a time. So it ends up at around 12% overall, and it takes over 4 hours to run compared to minutes for the other models.

To improve VADAR's performance on the LEGO benchmark, the system would need: (1) multi-image support in the execution engine so programs can reference and compare multiple images, (2) temporal reasoning capabilities to understand assembly sequences, and (3) more robust bounding box detection, since GroundingDINO frequently generates image-spanning boxes that undermine spatial analysis.